

System Documentation

Business Sustainability Management Platform

Group 9 — Five Guys

Document Type: System Documentation
Filename: Group_9_system_documentation.pdf
Project Focus: Web-based ESG and sustainability management platform
Date: April 24, 2026

Contents

Abstract

This document presents the system documentation for the Business Sustainability Management Platform developed by Group 9. The platform is intended to support practical ESG management in a multi-store business setting through integrated carbon tracking, waste management, sustainability reporting, alert monitoring, and governance-related ESG functions. Instead of relying on separate spreadsheets or isolated reports, the system provides a browser-based workflow that supports structured record keeping, clearer data visibility, and role-sensitive access.

The implemented system combines operational sustainability functions with higher-level governance features. Core modules include dashboard analytics, carbon and waste data recording, sustainability report generation, alert threshold management, supplier ESG assessment, ESG policy management, staff training records, compliance scoring, anomaly detection, user management, and audit logging. The platform also includes supporting functions such as OCR-assisted bill scanning, PDF export, mock fallback strategies for demonstration stability, and multi-role access control for administrators, headquarters managers, regional managers, and store staff.

Technically, the platform is implemented using Flask, Jinja2, Bootstrap, MySQL, and several supporting tools including Flask-Mail and WeasyPrint. In its current form, the system is complete enough to be tested and demonstrated, while still being lightweight enough for browser-based use and academic evaluation.

1 Introduction

1.1 Project Background

Environmental, Social, and Governance (ESG) concerns have become more important in business management [?], especially for organisations that operate across multiple physical locations and need more consistent sustainability oversight. In restaurant and coffee-shop style businesses, day-to-day sustainability work often includes repetitive operational tasks such as recording energy use, tracking waste generation, monitoring recycling performance, and preparing summary reports for management review. In many cases, however, these tasks are still handled through separate spreadsheets, manually prepared documents, or informal communication between branches. This makes long-term monitoring harder and can also reduce the transparency of sustainability claims.

The Business Sustainability Management Platform was developed to respond to this problem in a practical way. The project focuses on building a web-based platform that can bring sustainability-related operational data and governance functions into one interface. By integrating data collection, analytics, reporting, alerts, and ESG management, the system aims to improve traceability, consistency, and visibility for sustainability work across multiple stores. This direction is consistent with the reporting principles outlined in the GRI Standards [?] and the UN Sustainable Development Goal 12 [?].

1.2 Problem Statement

Before this platform was developed, sustainability-related activities in the target business scenario were difficult to monitor in a consistent and structured way. Carbon records, waste records, and reporting outputs could be created, but they were not easy to standardise across different stores. Management therefore lacked an efficient way to compare branch performance, detect unusual data patterns, monitor reporting coverage, or evaluate suppliers from an ESG perspective. At the same time, different users needed different permissions, which meant that a realistic solution required not only data-entry pages but also a clear access-control model.

The system was therefore designed to address three linked problems. First, it needed to provide a reliable operational layer for recording carbon and waste data. Second, it needed to turn these data into useful management outputs such as dashboards, reports, and alerts. Third, it needed to extend beyond operational monitoring into governance-oriented ESG management through supplier assessment, policy management, compliance scoring, and audit support.

1.3 Project Objectives

The main objective of the project is to create a practical ESG management platform that supports both sustainability operations and management decision-making. To achieve this objective, the project was designed around several concrete goals:

- provide structured carbon and waste tracking workflows for different business roles;
- support dashboard-based data visualisation and summary analysis;
- enable sustainability report generation and PDF export;
- provide alert and notification functions for management monitoring;
- integrate ESG governance features such as supplier assessment and policy management;
- support administrative and assurance functions including compliance scoring, anomaly detection, user management, and audit logging.

1.4 Project Planning and Scope Evolution

The project began with core sustainability functions—carbon tracking, waste management, dashboard review, and reporting—and gradually expanded through iterative refinement. The current version includes supplier ESG scorecard assessment, ESG policy management, compliance evaluation, anomaly detection, user administration, audit logging, OCR-assisted input, and alert-based monitoring, reflecting both the original sustainability direction and the later integration of governance and assurance functions.

1.5 Target Users

The platform is designed for four main user roles: administrators, headquarters managers, regional managers, and store staff. These roles differ in both their responsibilities and their access scope. Administrators and headquarters managers can oversee company-wide data and configuration. Regional managers focus on the stores within their region. Store staff primarily create and manage operational records for their own store. This role distinction shaped both the backend access-control logic and the frontend page visibility of the final system.

2 Technical Implementation

2.1 System Architecture and Technology Stack

The platform follows a web-based client-server architecture in which the presentation layer, application layer, and data layer are separated while still remaining lightweight enough for semester delivery. The backend is implemented with Flask [?] and organised through modular blueprints so that each business area, such as carbon tracking, waste management, reporting, alerts, ESG management, and administration, can be maintained as a relatively independent functional unit. This modular structure improves readability, simplifies route organisation, and makes later extension easier when new features are introduced.

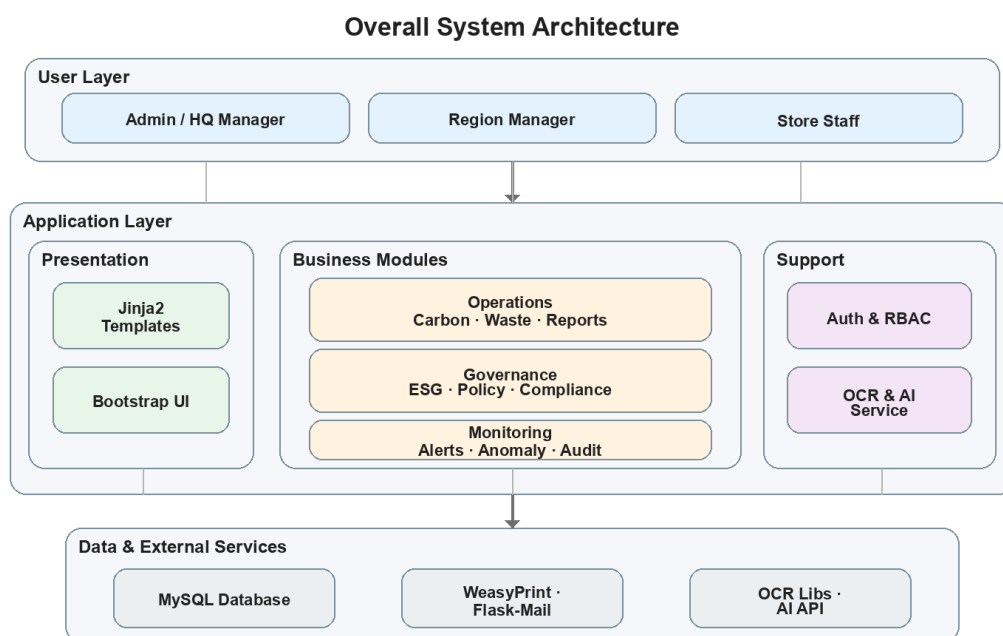


Figure 1: Overall System Architecture of the Business Sustainability Management Platform

On the frontend side, the project uses Jinja2 templates combined with Bootstrap to provide a browser-based interface with a consistent layout, reusable interaction patterns, and role-sensitive navigation. This approach suited the project because it allowed us to combine server-

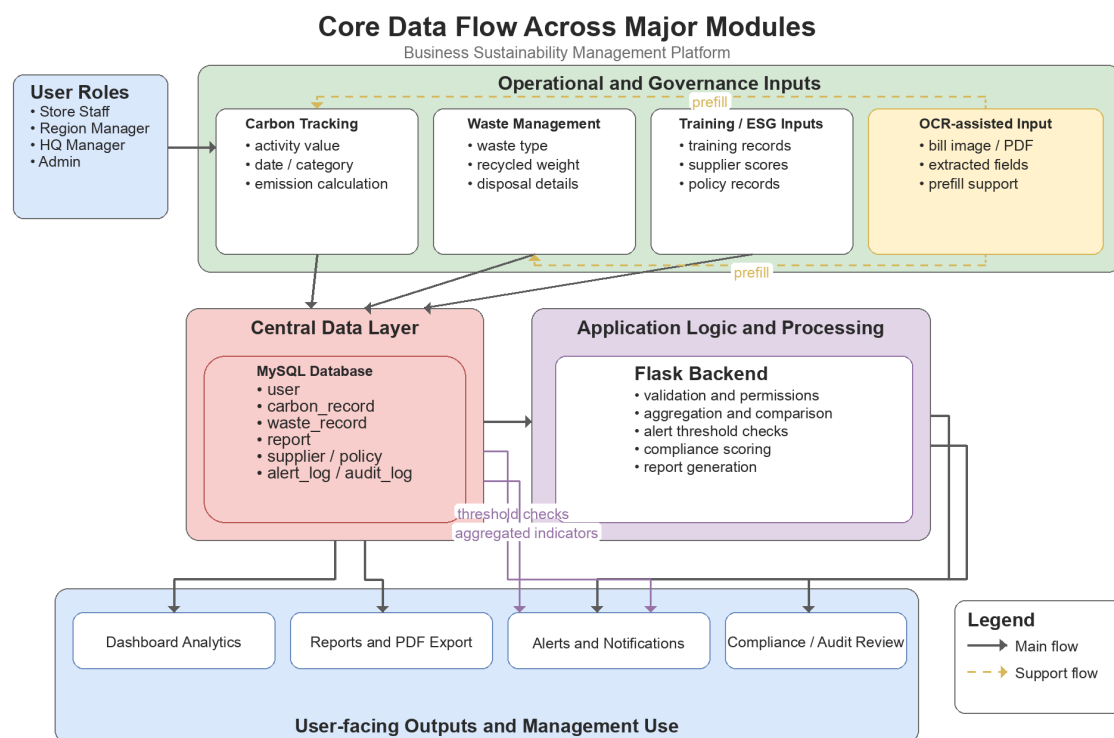


Figure 2: Core Data Flow Across Major Modules

rendered pages with lightweight dynamic behaviour without adding an unnecessary single-page-application layer. Shared templates provide consistent headers, sidebars, and visual structure across modules, while module-specific pages focus on tables, forms, filters, charts, and management interactions.

MySQL [?] is used as the persistent data layer for operational records, governance data, user information, alert logs, report content, and audit history. This choice supports structured querying, aggregation, and longer-term storage across several related modules. It is important because the platform does not only record isolated entries; it also reuses those records in comparisons, dashboards, reporting, compliance scoring, and anomaly detection. Supporting utilities extend this architecture in more specific ways. Flask-Mail is used for notification support, WeasyPrint is used for PDF report generation, and optional OCR-related libraries support scanned bill extraction when the environment permits. Together, these choices create a system that is not overly complex technically, but still broad in functionality.

At a high level, users interact with the system through browser-based pages and forms, submit or review data according to their role, and trigger backend logic that validates, stores, and transforms those data into management outputs. Submitted records then become part of a wider internal data flow: carbon and waste data feed dashboards and reports, alerts are triggered by thresholds linked to new records, compliance scoring aggregates indicators across modules, and governance pages draw on structured policy and supplier information. In this way, the architecture supports both day-to-day operational use and higher-level ESG management workflows within one platform.

Table / View	Category	Role in the System
company, region, store	Organisation	Define the three-level hierarchy (HQ → region → store) used for data isolation and scope-based access control
user	Identity	Store credentials, role (admin / hq_manager / region_manager / store_staff), and organisational scope for session-based authentication
emission_factor	Reference	Pre-seeded lookup of emission coefficients by category and sub-type, used to convert activity values into carbon kg during record creation
carbon_record	Operations	Record-level carbon data including activity value, calculated emissions, category, store, and date; feeds dashboards, reports, alerts, and comparisons
waste_category	Reference	Enumeration of waste types with recyclability flags; referenced by waste records for classification
waste_record	Operations	Record-level waste data including weights, disposal method, and store; contributes to recycling analysis, compliance, and reporting
report	Reporting	Persisted sustainability report with aggregated carbon and waste summaries, SDG indicators, optional AI commentary, and PDF export support
sdg_goal, company_sdg	Governance	UN SDG reference data and per-company alignment tracking with target values and progress percentages
alert_threshold	Monitoring	Configurable metric thresholds at company / region / store level for waste recycling rate, carbon MoM growth, and daily waste weight
alert_log	Monitoring	Triggered alert records with metric snapshots, read-state management, and optional email notification tracking
audit_log	Audit	Immutable log of user actions (login, create, update, delete, export) for traceability and compliance review
supplier	Governance	Supplier records with a 16-indicator ESG scorecard across carbon, waste, ethics, and reporting dimensions; rule-based scoring
esg_policy	Governance	Internal ESG policy documents with versioning, status lifecycle (draft / active / archived), and category tagging
v_carbon_monthly_summary, v_waste_monthly_*	View	Monthly aggregated carbon and waste summaries by store; used for dashboard charts and MoM trends
v_store_carbon_summary, v_store_waste_summary	View	Store-level monthly totals used for alert threshold evaluation (MoM growth, recycling rate)

Table 1: Key Database Entities and Their Roles

2.2 Authentication and Role-Based Access Control

The system uses username-and-password authentication with server-side session management. After successful login, the backend stores key identity and scope information in the session, including the current user’s role, company, region, and store context. This design fits a server-rendered Flask application because it keeps access control logic close to the business routes and avoids exposing sensitive authorisation state on the client side, in line with secure session management practices recommended by OWASP [?].

Role-based access control is implemented as a core design principle. Administrators and headquarters managers access company-wide data and configuration, regional managers are limited to their assigned region, and store staff are restricted to their own store. Within modules, store staff are subject to tighter edit and delete windows, while higher-level roles can configure alerts, manage users, access compliance views, and use governance features such as supplier ESG assessment and policy management. This multi-layered approach reflects real reporting hierarchies and operational accountability, making access control a structural feature of the system rather than a narrow security add-on.

Role	Scope	Carbon/Waste	Reports	Alerts	Key Management Functions
Admin	Company-wide	Full CRUD	Full	Config + review	Users, audit, compliance, anomaly, ESG management
HQ Manager	Company-wide	Full CRUD	Full	Config + review	Compliance, anomaly, ESG management, company users
Region Manager	Assigned region	Regional access	Regional	Review only	Compliance, anomaly, dashboard drilldown
Store Staff	Own store only	Own-store with time limits	Limited use	No config	No admin or governance configuration

Table 2: Role-Based Access Scope and Functional Permissions

2.3 Dashboard and Data Visualisation

The dashboard module transforms recorded operational data into summary indicators and management-oriented visualisations. It provides an overview of total carbon emissions, total waste, recycling performance, and report volume, while also supporting chart-based review of carbon trends, waste composition, and category-level breakdowns. Drilldown support enables users to move from aggregated company or regional indicators to more specific store-level views.

An additional analytical layer connects the dashboard to SDG 12 related indicators, including waste recovery, carbon efficiency, reporting coverage, and store data coverage. This makes the dashboard not only a display page but also a compact decision-support surface for sustainability oversight.

2.4 Carbon Tracking Module

The carbon tracking module supports the structured recording of carbon-related operational data and forms one of the most important operational foundations of the platform. In practice, this module is intended to capture recurring store-level activities that can be associated with carbon impact, such as electricity use or other emissions-relevant activities. Users enter activity values, dates, categories, and related contextual information, while the backend applies validation and calculation logic before committing the record to the database.

An important implementation detail is the use of emission-factor-based calculation. Instead of storing only a manually entered carbon total, the system links activity data with an emission context and calculates the resulting carbon value automatically, following the factor-based approach recommended by the GHG Protocol [?] and ISO 14064 [?]. This improves consistency and reduces the risk of arbitrary reporting. It also makes the module more reusable, because once the raw operational input has been recorded, the resulting carbon totals can be reused across several other parts of the platform, including dashboards, reports, comparisons, and compliance-related calculations. Figure ?? shows the core calculation logic.

The module also contains operational controls that make it more realistic than a basic data-entry form. Users can list, add, edit, and delete records, but these actions are constrained by both role scope and time-based business rules. In particular, store staff are subject to a limited

```

1 # Look up the emission factor from the reference table
2 factor = query_db(
3     'SELECT id, category, factor FROM emission_factor WHERE id = %s',
4     (factor_id,), one=True)
5
6 # Calculate total carbon: activity_value * emission coefficient
7 total_carbon = round(activity_value * float(factor['factor']), 4)
8
9 # Persist the record with both raw input and calculated output
10 insert_db(
11     'INSERT INTO carbon_record (company_id, store_id, user_id, '
12     'factor_id, category, activity_value, total_carbon, record_date, note) '
13     'VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)', (...))

```

Figure 3: Emission-Factor-Based Carbon Calculation (from `carbon.py`)

modification window, which reduces the likelihood of uncontrolled retroactive editing while still allowing reasonable correction of recent mistakes. This mirrors the practical idea that store-level data should be editable when first recorded but should not remain indefinitely open to change.

In addition to recording data, the module supports management-oriented analysis. Comparison functions enable month-on-month review and store-versus-region benchmarking so that managers can identify abnormal increases or performance differences between locations. Export support further improves practical usability by allowing data extraction beyond the live application view. The module is also strengthened by OCR-assisted input support, which helps prefill operational data from scanned bills or similar documents. For these reasons, the carbon module is not only a record-keeping page but a structured workflow that connects calculation, control, and analysis.

2.5 Waste Management Module

The waste management module provides a structured way to record, review, and analyse waste-related operational data. It captures information such as waste category, total waste weight, recycled quantity, source type, disposal method, disposal unit, notes, dates, and store linkage. These fields make the module more informative than a simple waste total, because they allow users and managers to understand not only how much waste was produced but also what kind of waste was generated and how it was handled.

Validation is particularly important in this module because waste data are often vulnerable to inconsistent manual entry. The backend therefore enforces required fields, sensible numeric constraints, and practical business rules such as ensuring that recycled weight does not exceed total recorded waste. Store scope restrictions are also applied so that users only interact with records within their permitted organisational boundary. Together, these rules help preserve data quality before the information is reused elsewhere in the system.

Like the carbon module, waste management supports record listing, creation, editing, and deletion. It also applies time-based restrictions for lower-permission users so that ordinary opera-

tional records remain manageable but not indefinitely mutable. This reflects a design principle used throughout the platform: operational flexibility is allowed, but long-term traceability is prioritised once data have become part of organisational monitoring.

The management value of this module extends beyond the raw record table. Waste data contribute to recycling analysis, trend comparison, dashboard summaries, report content, and compliance-related indicators. Comparison support enables users to review current performance against previous periods or wider organisational averages, while export support improves the practical portability of the data. Because waste handling is closely connected with responsible consumption and recovery performance, this module also plays a significant role in the system's connection to broader ESG and SDG-oriented reporting.

2.6 Sustainability Reporting Module

The sustainability reporting module is one of the clearest examples of how the platform transforms operational records into management-ready outputs. Rather than requiring users to manually rewrite carbon and waste statistics into separate documents, the system aggregates data within a selected time range and generates structured report content directly from stored records. This allows reporting to remain connected to the same data foundation used elsewhere in the application and reduces duplication of work.

From an implementation perspective, the module performs several linked tasks. It accepts a reporting period, queries the relevant carbon and waste data, derives summary values, and then stores the resulting report as a persistent record in the database. This means that a report in the platform is not just a temporary generated page; it becomes a reusable artefact that can later be listed, viewed in detail, exported, or deleted. This approach improves traceability because reporting history can be examined over time instead of being regenerated from scratch each time.

PDF export is a major part of this module. The system first renders report content through a Jinja2-based HTML structure and then uses WeasyPrint to produce a downloadable PDF. This design fits the wider architecture well because it reuses templating logic already present in the web layer while still generating a formal document output suitable for submission, review, or demonstration. The report content is also connected with SDG 12-related indicators so that the final output contains not only raw totals but also more interpretive sustainability information.

Another important extension is AI-assisted report commentary. The system supports a real external API pathway for generating narrative ESG comments while also preserving a mock fallback path for development and testing contexts. This fallback strategy is significant because it allows the report module to remain demonstrable even when external services are unavailable. In the final system, the reporting module therefore operates as a bridge between stored operational data, analytical interpretation, formal PDF export, and optional AI-enhanced commentary.

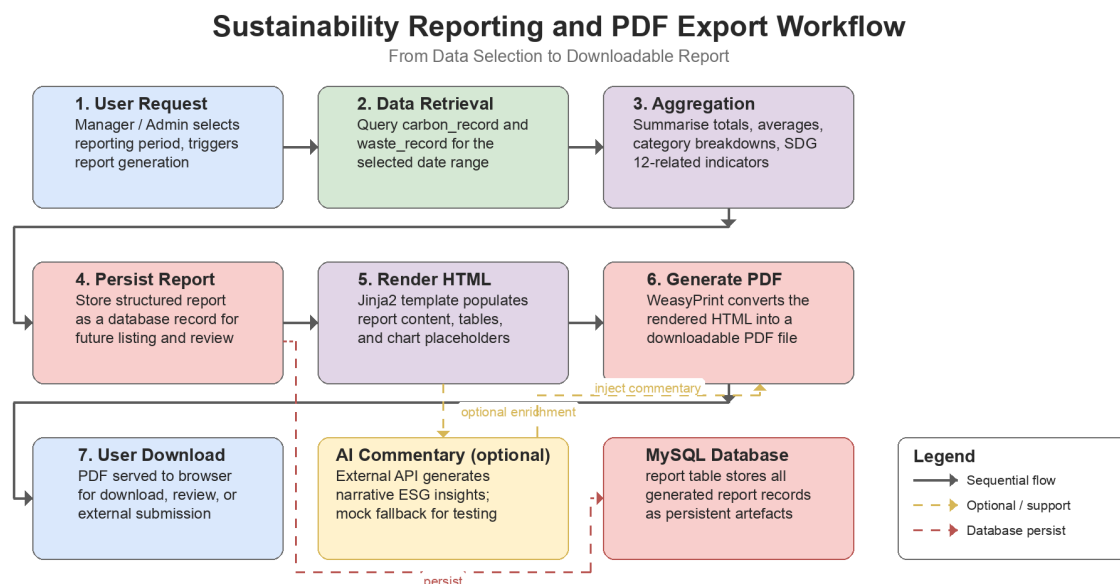


Figure 4: Sustainability Reporting and PDF Export Workflow

2.7 Alerts and Notification Mechanism

The alert subsystem provides threshold-based monitoring for sustainability performance. Users with appropriate permissions can define thresholds for selected metrics at company, region, or store level. When new operational data are processed, relevant alert logic can be triggered and recorded for later review.

The module supports alert logs, unread-count indicators, and read-state management, making it suitable for ongoing managerial monitoring rather than one-time reporting only. Email-based notification support is also integrated through Flask-Mail, while fallback behaviour helps preserve usability in test or demo environments where full mail configuration may not be available.

2.8 ESG Management Module

The ESG Management section combines supplier assessment and ESG policy management under a unified interface. This reflects the fact that ESG management in practice involves both operational evaluation of partners and the maintenance of governance rules within the organisation. Structurally, placing these functions together also improves the clarity of the system because it distinguishes governance-oriented ESG work from day-to-day operational record entry.

This part of the system is particularly important because it shows that the platform is not limited to measuring internal store activity. It also supports broader organisational responsibilities such as evaluating external partners and maintaining internal ESG policy records. In other words, this module extends the system from environmental operations into governance-oriented sustainability management.

2.8.1 Supplier ESG Assessment

The supplier module manages supplier records while also evaluating them through a rule-based ESG scorecard. The current model uses four dimensions—carbon, waste, ethics, and reporting—and each dimension contains four sub-indicators scored as 0, 50, or 100. Weighted aggregation is then used to calculate both dimension-level scores and an overall grade. This design was chosen deliberately. A simple manual score field would be too opaque, while a data-hungry black-box model would not be realistic for the project context. The implemented scorecard therefore offers a middle ground: it is structured, explainable, and practical.

The use of 0, 50, and 100 as discrete scoring values makes the model easier to interpret during data entry and later review. It encourages users to think in terms of whether a supplier does not meet, partially meets, or clearly meets a given ESG-related expectation. Because the model is rule-based, score changes can be explained directly by the selected sub-indicators rather than hidden behind inaccessible logic. This is especially useful for academic demonstration, where transparency and justification are more important than claiming unrealistic predictive sophistication.

Dimension	Example Sub-indicators	Score Options	Purpose
Carbon	Disclosure, reduction target, carbon monitoring, supporting evidence	0 / 50 / 100	Evaluates whether the supplier tracks and improves carbon-related performance
Waste	Waste reduction, recycling practice, disposal control, waste reporting	0 / 50 / 100	Captures resource-efficiency and waste-handling responsibility
Ethics	Labour practice, compliance awareness, traceability, ethical policy support	0 / 50 / 100	Reflects social and governance expectations in supplier behaviour
Reporting	ESG disclosure, consistency, completeness, review readiness	0 / 50 / 100	Measures the supplier's reporting transparency and management maturity

Table 3: Structure of the Supplier ESG Scorecard

From a workflow perspective, the supplier module supports adding, editing, listing, and deleting supplier records, while automatically recalculating ESG scores whenever data are submitted. Figure ?? illustrates the weighted aggregation logic. This removes dependence on manually entered total scores and ensures that the overall grade remains aligned with the selected sub-indicator values.

2.8.2 ESG Policy Management

The ESG policy module manages internal policy records such as policy category, version, status, effective date, and content. These records are presented within the same ESG Management area as supplier assessment, which creates a stronger governance-oriented workflow. In practical terms, this allows the system to represent both ESG evaluation and ESG policy maintenance within one management space.

Although the policy module is structurally simpler than supplier scoring, it is important for

```

1 OVERALL_WEIGHTS = {'carbon': 0.30, 'waste': 0.30,
2                   'ethics': 0.20, 'reporting': 0.20}
3
4 def _build_scorecard(data):
5     # Per-dimension weighted score (each indicator: 0/50/100)
6     for dimension, fields in DIMENSION_FIELDS.items():
7         values = [normalized[f] for f in fields]
8         weights = OVERALL_WEIGHTS[dimension]
9         dimension_scores[dimension] = round(
10             sum(v * w for v, w in zip(values, weights)))
11     # Overall weighted score across all dimensions
12     overall = round(sum(
13         dimension_scores[k] * OVERALL_WEIGHTS[k]
14         for k in OVERALL_WEIGHTS))
15     return {'dimension_scores': dimension_scores,
16           'overall_score': overall,
17           'overall_grade': _grade_from_score(overall)}

```

Figure 5: Supplier ESG Scorecard Weighted Aggregation (from `supplier.py`)

completeness. ESG management in a realistic organisation does not only involve measuring outcomes; it also involves maintaining policy documents, version control, implementation status, and effective dates. By including these fields in the system, the platform supports a clearer connection between governance rules and operational sustainability practices.

2.9 Training and Compliance Evaluation

Training and compliance functions extend the system from operational data collection into organisational improvement and assurance. The training module stores sustainability-related training records, course types, completion data, scores, and summary statistics. These records help represent staff capability development as part of ESG management rather than as a disconnected administrative activity. In other words, the system recognises that sustainability performance does not depend only on recorded outcomes, but also on whether staff are being prepared to carry out related practices consistently.

The compliance score module builds on this wider perspective by aggregating multiple ESG-relevant dimensions, including carbon management, waste and recycling, staff training, reporting frequency, and alert resolution. This design is useful because it combines operational and managerial signals rather than relying on a single metric. For example, an organisation might record waste data consistently but perform poorly in training coverage or alert handling. A broader compliance score therefore provides a more balanced summary of the organisation's ESG readiness and discipline.

By calculating a weighted total score and final grade, the platform converts distributed activity across multiple modules into a compact summary that can be read quickly and compared over time, acting as a synthesis layer between operational data quality and governance-oriented performance.

Dimension	What It Measures	Main Data Source
Carbon Management	Consistency and quality of carbon-related operational records	Carbon tracking records and comparisons
Waste and Recycling	Waste handling quality, recycling performance, and recovery-related behaviour	Waste management records and recovery metrics
Staff Training	Participation, completion, and coverage of sustainability-related training activity	Training records and summary statistics
Reporting Frequency	Whether sustainability reporting is being produced regularly and in usable form	Stored report records and reporting history
Alert Resolution	Whether monitored issues are noticed, logged, and followed through at management level	Alert thresholds, alert logs, and review status

Table 4: Dimensions Used in ESG Compliance Evaluation

2.10 Advanced Management and Support Functions

The platform includes additional modules for data quality assurance, user administration, accountability, and input convenience, each described below.

2.10.1 Anomaly Detection

The anomaly detection module applies two complementary strategies to flag unusual carbon and waste records. The statistical approach uses Z-score analysis: for a configurable look-back window (default 90 days), records are grouped by category, and any record whose Z-score exceeds the threshold (default 2.0) is flagged as medium ($|z| \geq 2$) or high ($|z| \geq 3$) severity. The rule-based approach flags structural impossibilities directly—zero or negative carbon activity values, and waste records where recycled weight exceeds total weight. Table ?? summarises both methods.

Method	Applies To	Trigger Condition	Severity
Z-score (statistical)	Carbon	Per-category total_carbon deviates $\geq$ threshold σ from mean	Medium ($ z \geq 2$); High ($ z \geq 3$)
Z-score (statistical)	Waste	Per-category weight_kg deviates $\geq$ threshold σ from mean	Medium ($ z \geq 2$); High ($ z \geq 3$)
Rule-based	Carbon	Activity value ≤ 0	High
Rule-based	Waste	Recycled kg $>$ total weight kg	High

Table 5: Anomaly Detection Methods and Trigger Conditions

The Z-score detection logic is shown in Figure ??.

2.10.2 User Management

The user management module enables administrators and headquarters managers to create, update, delete, and reset passwords for user accounts within their organisational scope. All operations are scope-restricted: administrators manage users platform-wide, while headquarters

```

1 def _zscore_anomalies(records, value_key, label_fn, threshold=2.0):
2     values = [float(r[value_key] or 0) for r in records]
3     if len(values) < 3:
4         return []
5     mean = sum(values) / len(values)
6     std = math.sqrt(sum((v - mean)**2 for v in values) / len(values))
7     if std == 0:
8         return []
9     anomalies = []
10    for r, v in zip(records, values):
11        z = (v - mean) / std
12        if abs(z) >= threshold:
13            severity = 'high' if abs(z) >= 3.0 else 'medium'
14            anomalies.append({'record_id': r['id'], 'z_score': round(z, 2),
15                             'severity': severity, ...})
16    return anomalies

```

Figure 6: Z-Score-Based Anomaly Detection (from `anomaly.py`)

managers are limited to their own company. The system validates username uniqueness, password strength, and role assignment against the four defined system roles, and stores passwords using secure hashing. Safeguards prevent users from deleting their own account or modifying accounts across company boundaries. Every management action is recorded in the audit log with the responsible user, target, and IP address.

The interface provides a tabular user list with inline controls and dynamically populated drop-downs for region and store assignments, ensuring role and scope selections remain consistent with the organisational hierarchy in the database.

2.10.3 Audit Logging

The audit logging module records significant user actions—logins, record creation, updates, deletions, exports, and administrative operations—into an immutable log table. Each entry captures the acting user, action type, target entity, a detail string, IP address, and timestamp. The dedicated management page supports filtering by action type, target type, and date range, and a statistics endpoint aggregates action counts over the past seven days. Visibility respects role-based isolation: administrators see all entries, while headquarters managers see only their company’s entries. Log entries are written only for confirmed operations, and sensitive data such as passwords are excluded.

2.10.4 OCR-Assisted Input

The platform supports OCR-assisted bill scanning to extract fields such as category, activity value, date, and supplier name from uploaded images or PDFs, which are then used to prefill data entry forms. The feature uses a fallback-aware design: when OCR libraries are available, uploads are processed through the extraction pipeline; otherwise the system falls back gracefully, keeping the rest of the platform fully functional.

2.11 Database, Migration, and Deployment Support

The platform depends on a structured MySQL schema that stores user, operational, reporting, governance, and monitoring data. Because the system evolved throughout the semester, migration support became an important part of implementation. Migration scripts are used to update schema structure in an incremental and repeatable way, including support for newly introduced ESG scorecard fields and related data changes.

The project also uses fallback strategies to improve robustness in evaluation settings. Mock data and fallback execution paths are available for selected modules such as supplier records, policy content, AI commentary, OCR input, and email notification. This helps maintain functional demonstrations even when external services or optional dependencies are unavailable.

3 Use of Generative AI

Generative AI tools were used during this project as supporting assistants rather than as replacements for design or implementation decisions. In practice, AI was helpful in four main areas: (1) coding productivity—suggesting implementations, refactoring ideas, and bug-fix directions; (2) debugging—interpreting error messages and identifying likely causes; (3) documentation—improving wording, reorganising content, and expanding technical descriptions; and (4) design communication—turning rough ideas into clearer specifications that could be compared with the actual codebase.

However, we found that AI output could not be trusted directly. In a Flask project with many blueprints, role-based permissions, and business rules, AI-generated code sometimes ignored the actual database schema, used wrong variable names, or described features that had not been implemented. Similarly, AI-written documentation could blur the line between implemented functions and future extensions, or make a feature sound more mature than it really was. These risks were especially problematic for system documentation, which must describe the real system accurately. We therefore checked all AI-assisted output manually: code suggestions were compared against existing routes, templates, and schema; documentation was verified against actual pages and permission logic; and LaTeX changes were confirmed through local compilation.

Overall, AI improved our development speed and documentation efficiency but did not reduce the need for human judgement. The most reliable approach was to use AI as a reviewer and writing assistant on tasks where we already understood the target module well enough to evaluate the result critically. Prompts that were too open-ended or lacked project context produced more hallucinations. For that reason, our team treated AI as a productivity tool rather than a replacement for technical responsibility, and final acceptance always depended on manual verification.

4 Groupwork

Our project followed a work-package structure, so each member had a main area to focus on, while still helping with integration and later revisions across the system. This worked reasonably well for our team because it gave everyone clear responsibilities without making the work completely separate. As the project developed, several modules became more connected than we expected at the beginning, especially around ESG management, reporting, compliance, and integration. Because of that, the final contribution pattern was not only based on the original work-package plan, but also on the adjustments we made during implementation.

In terms of collaboration workflow, the team used Git for version control and coordinated through regular online meetings. Code reviews were conducted informally within functional pairs (e.g. backend and frontend developers reviewing each other’s integration points). Documentation tasks were split between two members at a time, with one person drafting content and the other reviewing against the actual codebase. This approach helped reduce inconsistencies and ensured that written descriptions matched the implemented system rather than an idealised version.

WP	Lead	Focus Area	Key Deliverables
WP1	Yao Tianren	Requirements & Design	User analysis, architecture, database design, low-fidelity prototypes
WP2	Liu Haopu	Backend Development	API routes, validation, business logic, schema, migrations
WP3	Li Sihang	Frontend Foundation	Framework setup, auth UI, reusable components, role-sensitive navigation
WP4	You Haoyang	Business Frontend	Carbon/waste/report pages, charts, dashboard, PDF export integration
WP5	Lu Yuan	Integration & Testing	End-to-end flow, compatibility, deployment, performance optimisation
WP6	Liu Haopu	Delivery & Documentation	System/user docs, test accounts, VM deployment, presentation

Table 6: Work-Package Structure and Responsibilities

4.1 Yao Tianren

Yao Tianren was mainly responsible for WP1, which covered requirements analysis and overall system design. His main contribution was to help define the scope of the project and turn broad sustainability goals into a system structure that could realistically be implemented within one semester. This included analysing the likely needs of business users, identifying key scenarios for carbon tracking, waste management, and sustainability reporting, and helping shape the project into a multi-role ESG management platform rather than just a few separate record pages.

Besides the early planning work, he also contributed to the technical direction and the overall modular structure of the platform. His work included system architecture planning, functional decomposition, database structure planning, and low-fidelity design preparation. These early design choices influenced later modules such as ESG policy management, supplier assessment,

and compliance-related workflows. He also took part in reviewing design artefacts and discussing revisions so that later implementation still matched the original aims of the project.

In the later stages, he participated in cross-module design discussions and contributed to the refinement of the compliance scoring model and the ESG governance module layout. His architectural understanding also helped resolve design questions when new features were added beyond the original scope.

4.2 Liu Haopu

Liu Haopu was mainly responsible for WP2 and WP6, covering backend development as well as project delivery and final documentation. His main contribution was building the backend foundation of the platform, including database instance preparation, schema construction, API design, validation logic, and business-rule handling across the main functional modules. This included account management, carbon and waste workflows, sustainability reporting, alert processing, ESG management, and other core backend services.

His work also extended into later-stage modules that became important in the final version of the platform, such as report generation support, PDF export integration, supplier ESG scorecard logic, policy-related backend support, and migration handling for an evolving database structure. In the final delivery stage, he also worked on system documentation, user-facing setup guidance, testing account preparation, deployment-related tasks, and presentation materials. This made his role closely connected with both implementation and final handover.

Key technical contributions included the weighted supplier scorecard algorithm, the multi-dimensional compliance scoring engine, the emission-factor-based carbon calculation pipeline, and the database migration framework for incremental schema evolution.

4.3 Li Sihang

Li Sihang was mainly responsible for WP3, which focused on the frontend foundation of the system. His main task was to establish the structural base of the interface layer so that later business modules could be developed in a more consistent way. This included frontend framework setup, page organisation, routing-related coordination in the Flask/Jinja environment, reusable UI component preparation, request integration patterns, and general interface consistency.

He also contributed to role-sensitive frontend behaviour and the shared interaction structure of the system, including navigation layout, shared forms, modal-based interaction patterns, and consistent visual styling. These parts were important because the system supports several user roles with different permissions and visible modules. By building a stable and reusable frontend foundation, he made it easier to integrate later pages such as ESG management, compliance review, and administrative features without major redesign of the shared UI.

Specific deliverables included the `apiFetch` helper for authenticated JSON requests, the shared

sidebar and navigation template with role-conditional visibility, the login and registration flow, and the reusable modal and alert components used across all business modules.

4.4 You Haoyang

You Haoyang was mainly responsible for WP4, which focused on the implementation of core business-facing frontend modules. His contribution was mainly to turn business requirements into user-facing workflows across the main sustainability functions of the platform. This included carbon-tracking pages, waste-management interfaces, reporting-related interaction, and the visual presentation of data through dashboard components and page-level integration.

His work also supported the interactive quality of the system by improving the usability of forms, tables, visual summaries, and role-relevant workflows. As the project developed, this role also extended to the integration of additional operational interfaces connected with report generation, ESG management views, and other business-oriented pages. Through this work, the frontend became more than a set of static pages and moved closer to a coherent interface for daily sustainability management and management review.

Notable technical work included the ECharts-based dashboard visualisations (carbon trend, waste composition, category breakdown charts), the report preview and PDF download flow, the carbon and waste comparison modals, and the supplier scorecard form with dynamic score calculation feedback.

4.5 Lu Yuan

Lu Yuan was mainly responsible for WP5, which focused on system integration, testing, and deployment-oriented refinement. His main task was to connect the database, backend logic, frontend pages, and business workflows into a usable end-to-end platform. This included identifying module interaction problems, supporting compatibility and integration checks, and improving the overall stability of the system as features were brought together.

This contribution became especially important in the later stages of development, when the project expanded to include compliance scoring, anomaly detection, alert review, audit support, and other management-oriented functions. Integration and testing helped make sure these modules were not treated as separate experiments, but as parts of one coherent system. In addition, his role also covered deployment-oriented testing, performance-related refinement, and support for final validation in an environment suitable for demonstration and remote review.

Specific contributions included cross-browser compatibility testing, end-to-end workflow validation for all four user roles, network adaptation for remote access from Ireland, performance profiling of database queries, and coordination of the final deployment onto the course VM.

5 Conclusion

The Business Sustainability Management Platform has developed into a comprehensive system that supports operational record management, role-based access control, dashboard analytics, report generation, alert monitoring, ESG governance workflows, training support, compliance evaluation, anomaly detection, user administration, and audit tracking. One of its main strengths is the combination of everyday business workflows with broader governance and assurance functions: carbon tracking and waste management provide the operational data foundation, while reporting, compliance scoring, supplier assessment, and policy management extend these records into meaningful management processes.

Overall, the current implementation provides a credible and testable ESG platform that reflects both technical development work and business-oriented sustainability management concerns. Although there is room for future extension—such as deeper external data integration or more advanced analytical models—the present version demonstrates a coherent, functional, and well-scoped system suitable for user testing, presentation, and further improvement.

References

- [1] Global Reporting Initiative, *GRI Standards*, GRI, Amsterdam, 2021. Available: <https://www.globalreporting.org/standards/>
- [2] R. G. Eccles, I. Ioannou, and G. Serafeim, “The Impact of Corporate Sustainability on Organizational Processes and Performance,” *Management Science*, vol. 60, no. 11, pp. 2835–2857, 2014.
- [3] Pallets Projects, *Flask: Web Development, One Drop at a Time*, 2023. Available: <https://flask.palletsprojects.com/>
- [4] World Resources Institute and World Business Council for Sustainable Development, *The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard*, Revised Edition, 2004. Available: <https://ghgprotocol.org/>
- [5] Oracle Corporation, *MySQL 8.0 Reference Manual*, 2024. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [6] International Organization for Standardization, *ISO 14064-1:2018 – Greenhouse gases – Part 1: Specification with guidance at the organization level for quantification and reporting of greenhouse gas emissions and removals*, ISO, Geneva, 2018.
- [7] United Nations, “Goal 12: Ensure sustainable consumption and production patterns,” *Sustainable Development Goals*, 2015. Available: <https://sdgs.un.org/goals/goal12>
- [8] OWASP Foundation, *OWASP Top Ten – 2021*, 2021. Available: <https://owasp.org/www-project-top-ten/>